PFP V: A Practitioner's Guide to Checked Proposals

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Practitioner guide · Author-review revision

Code baseline: 52cb5457f6c485ba548cfd292f6012a941ed22cf

Purpose and reader outcome

This guide takes one small inconsistent graph through a complete session: declare the instance, propose a change, interpret a refusal, admit a repair, check the result, and restore an earlier state. It is a practitioner companion to the Proposal Fidelity Protocol (PFP) research series and its legacy Symbol Game implementations. It adds no new theorem or measured model-performance result.

A **proposal** is a candidate supplied by a person, script, or model. A **host** is the program that applies the declared checks. A **receipt** records what a particular execution checked and returned. These roles matter because a plausible proposal can be wrong, while a successful check can establish less than its surrounding prose suggests.

The first session uses a scripted proposer. No model account, API key, or live conversation is needed. You should finish able to explain both an accepted move and the limits of its acceptance.

1. Declare the instance before solving it

Suppose three events have proposed offsets on one linear time axis:

- Event b is one unit after event a.
- Event c is one unit after event b.
- Event a is one unit after event c.

This is a mathematical offset example. There is no wraparound clock or recurring-day convention. Declare those conventions differently and you have a different problem.

Represent the events by vertices a, b, c and the statements by directed edges. The ordered claim vector is

$$c = (c_{ab}, c_{bc}, c_{ca}) = (1, 1, 1).$$

A **vertex potential** assigns a number f_a, f_b, f_c to the vertices. Its edge differences are

$$df = (f_b - f_a,\ f_c - f_b,\ f_a - f_c).$$

The differences always sum to zero: each vertex value appears once with each sign. The supplied claims sum to three. Consequently, no potential satisfies all three unchanged claims.

For the common series example, declare edge weights $w = (1, 2, 3)$. A weight is the penalty for leaving one unit of discrepancy on that edge. It is a modeling choice, not evidence that one event is more truthful. The complete instance therefore includes the vertices, directed edges, edge order, exact claims, weights, and permitted changes.

2. Separate a residual from a changed claim

For a chosen potential, the **residual** is $r = c - df$. Its weighted discrepancy is

$$D_c(f) = |r_{ab}| + 2|r_{bc}| + 3|r_{ca}|.$$

The **quotient minimum** is the smallest such discrepancy over all potentials:

$$\Phi_w(c) = \min_f D_c(f).$$

The notation distinguishes the score at one chosen potential from the best possible score for unchanged claims. A poor potential can make $D_c(f)$ larger without changing $\Phi_w(c)$.

Take $f = (0, -2, -1)$. Then $df = (-2, 1, 1)$, so $r = (3, 0, 0)$ and $D_c(f) = 3$. For every potential, the residual still sums to three. Because all weights are at least one,

$$D_c(f) \geq |r_{ab}| + |r_{bc}| + |r_{ca}| \geq |r_{ab} + r_{bc} + r_{ca}| = 3.$$

The displayed potential attains this lower bound. This short argument establishes the minimum for this instance without requiring trust in a solver.

Two operations now have different meanings. Choosing that potential concentrates the original discrepancy on one edge; the original cycle sum remains three. Replacing the claims themselves by $c' = (-2, 1, 1)$ produces a new consistent instance: $c' = df$ and $\Phi_w(c') = 0$. The governor demonstration performs this second operation through `step_cut(proposed_claims=...)`. Acceptance says the changed claims satisfy the declared graph check. It does not establish that the changed event statement is factually correct.

3. Choose the executable path

The supported tutorial in this package is code/quickstart.py. Its Python wrapper sends exact graph inputs to the pinned Rust calculator. The repository also contains a separate historical root `quickstart.py`; the commands here intentionally name the package file.

Use Python 3.10 or newer. The package is self-contained: it carries copies of the two modules it needs from the repository, the native graph adapter code/native_functional.py and the Paper 004 graph governor code/l1_symbol_game_governor.py. Each copy begins with a header that names its source path and commit, gives the SHA-256 of the source bytes, and lists the local changes. Apart from its header, the adapter differs from the source baseline only in import order. The governor copy is later than the baseline, and this package patches it in two places: it uses the native calculator only when one is configured and then lets every native refusal propagate, computing the minimum by exact Python enumeration otherwise (Section 4), and it refuses a CUT that omits a declared edge (Section 5). Inside a checkout of the source repository, code/tests/test_vendored_sources.py compares both copies with the recorded Git objects; elsewhere that comparison skips and says why.

The native calculator itself is not part of this package. Build the `solve_graph_phi1` executable from the reviewed source with the repository's Rust workspace, on any platform:

```
cargo build --locked -j 1 --manifest-path omega_engine/crates_rs/Cargo.toml -p
    omega_functional --bin solve_graph_phi1
```

The following Windows PowerShell setup, run from the repository root, assumes the reviewed executable already exists at the shown path:

```
$env:OMEGA_FUNCTIONAL_GRAPH_BIN = (Resolve-Path omega_engine/crates_rs/target/debug/
    solve_graph_phi1.exe).Path
$env:OMEGA_FUNCTIONAL_GRAPH_SHA256 = (Get-FileHash -Algorithm SHA256 $env:
    OMEGA_FUNCTIONAL_GRAPH_BIN).Hash.ToLowerInvariant()
```

Use the actual output path if Cargo has a custom target directory. Other platforms use their native executable name. The digest pins the executable bytes used by the adapter; it is not an independent attestation that those bytes were reproducibly built from the intended source.

Now, from this package's directory, run (on Windows, `py -3.12` in place of `python`):

```
python code/quickstart.py --non-interactive
python code/play_demo.py
```

The repository's functional-boundary notes describe the native setup at the baseline. The transport implementation is native_functional.py. A missing executable, missing or mismatched digest, native refusal, or malformed native response prevents an accepted native result. Treat that outcome as an environment or evidence issue to resolve before relying on the host calculation.

Offline arithmetic when the native calculator is unavailable

You can still understand and check the finite argument. Run this ordinary Python snippet, or evaluate it by hand:

```
from fractions import Fraction as Q

claims = (Q(1), Q(1), Q(1))
potential = (Q(0), Q(-2), Q(-1))
edge_differences = (
    potential[1] - potential[0],
    potential[2] - potential[1],
    potential[0] - potential[2],
)
residual = tuple(c - d for c, d in zip(claims, edge_differences))
assert edge_differences == (Q(-2), Q(1), Q(1))
assert residual == (Q(3), Q(0), Q(0))
assert sum(edge_differences) == 0
assert sum(residual) == 3
```

This is an arithmetic worksheet. It neither calls the native adapter nor emits a canonical functional receipt. Its lower-bound explanation is the argument in Section 2. The package test code/tests/test_offline_worksheet.py runs this block as printed. Record a native run as unavailable until the required calculator can execute.

4. Follow the complete session

play_demo.py imports the package's copy of the graph governor. Its graph uses weights (1,2,2), a legacy variant of the common series example's (1,2,3). Both have the same unique cheapest edge, minimum three, and displayed repair. The all-zero potential costs five in the script and six in the common example; those are scores of one potential, not the quotient minima.

When OMEGA_FUNCTIONAL_GRAPH_BIN is set, the governor asks that native calculator, and any refusal or error it returns stops the step; no Python value is substituted. When it is not set, the governor computes the quotient minimum by exact Python enumeration of spanning-tree potentials. Without a configured calculator the session therefore still runs, but its printed values are Python enumeration results, not native results. Edge weights must be exact integers or fractions on both paths; floating-point and Boolean weights are refused. The package test code/tests/test_play_demo.py checks the table below on the enumeration path, and again through the native calculator when pins are configured.

Step	Proposed action	Expected observation
Declare	Create vertices a, b, c and weighted edges	The host has a finite declared carrier
ASSERT	Install claims $1, 1, 1$	The original quotient minimum is 3
CUT	Propose replacement $5, 0, 0$	`DefectIncrease`, not admitted
CUT	Propose replacement $-2, 1, 1$	`ExactRepair`, admitted, 3 to 0
VERIFY	Request `ZERO_OBSTRUCTION`	The host checks its stored claims and admits the check
RETRACT	Restore the saved post-ASSERT snapshot	The original claims and minimum 3 return

The bad proposal sums to five. Its optimum is five by the same lower-bound argument, with all discrepancy placeable on the weight-one edge. The host refuses an increase from three to five. A refused proposal does not replace the active claims.

The good proposal changes the cheapest edge from one to minus two. It supplies a consistent replacement. The demonstration saves a snapshot before either CUT and uses that snapshot's actual identifier when retracting. Do not guess a snapshot identifier from a printed step count.

ASSERT installs the starting problem; it is not a claim that every initialization step decreases discrepancy. The governor's CUT checks permit non-increase. A tied score can therefore return `IdempotentStationary`. Strict progress and finite termination of an arbitrary search require additional conditions; repeating an admitted stationary move is not a solution strategy.

5. Read refusals as information

A refusal identifies a failed check. First determine which check ran and which input caused it. Then change the relevant proposal or declare a new instance when the intended problem requires different rules.

Observed result	Meaning in this implementation	Next operator action
`CarrierViolation`	A proposed endpoint is outside the declared vertices	Correct the endpoint or explicitly formulate another carrier
`MoveViolation`	A proposed edge is not one of the declared edges	Use a declared relation or revise the instance specification

Observed result	Meaning in this implementation	Next operator action
`IncompleteProposal`	A CUT's replacement claims omit a declared edge or give one a value that is not an exact integer or fraction	Supply exactly one exact value for every declared edge; an omitted edge is never read as a zero claim
`InvariantViolation`	A prohibited input, such as a self-edge, reached the gate	Inspect the exact failed input condition
`DefectIncrease`	The proposed score exceeds the current score under the selected CUT mode	Compute why it increased before proposing another change
`IdempotentStationary`	A permitted move left the checked score unchanged	Identify a new argument or record the lack of progress
`VerificationFailed`	The requested obligation did not pass	Inspect that obligation, rather than treating a different successful check as a substitute

The native transport can also fail before a governor verdict exists. Keep the transport exception and native response, if any, with the run record. A missing calculation is not a zero discrepancy.

Use integers and `Fraction` inputs for mathematical values. The tutorial rejects floats and Boolean values at its mathematical input boundary. In particular, Python's treatment of `True` as an integer-like value must not turn it into a declared numerical witness.

6. Reproduce a result and inspect its custody

A useful reproduction record names the source revision, exact input, command, Python version, native executable digest, exit status, and output. If files change during a run, identify which version each observation belongs to. Preserve refused cases alongside admitted ones: they show whether the gate discriminates.

Run the package checks from this package's directory, one test file per command:

```
python -m pytest code/tests/test_offline_worksheet.py -q
python -m pytest code/tests/test_play_demo.py -q
python -m pytest code/tests/test_quickstart_native.py -q
```

The fixture consumes only explicitly supplied OMEGA_FUNCTIONAL_GRAPH_BIN and OMEGA_FUNCTIONAL_GRAPH_SHA256. It discovers no checkout, builds no program, and creates no pin. Missing configuration skips, with that reason, every test that requests native execution. A malformed digest, unreadable configured executable, or digest mismatch fails those tests with an explicit message. The worksheet, the offline session, the refusal, exact-input, parser, and scaffold checks, and the vendored-source header checks still run without native configuration. The commands are replay instructions, not a reported fresh pass.

The existing execution receipt, cross-domain receipt, and series completion receipt are historical artifacts until their source and executable bindings are reconciled with a new run. A SHA-256 digest can bind bytes. It does not establish their truth, authenticate an unsigned producer, or supply a missing experiment. A restricted sorted-JSON encoding also needs its own stated contract; it should not be called full RFC 8785 conformance without checking that specification.

7. Map a new problem carefully

Before adapting the example, write down what a vertex means, what an edge value measures, why the edge-difference relation is appropriate, and which changes are allowed. Supply units and direction conventions. Decide whether changing a claim is an acceptable correction or would replace an observation you must preserve.

For example, measured offsets might need an uncertainty model rather than unrestricted replacement. A dependency relation might be Boolean rather than an additive quantity. Encoding either as this graph is a modeling step that requires justification. Renaming vertices to "currency," "permission," or "quantum state" does not supply that justification.

The package's four labeled graph scenarios illustrate repeated use of the finite interface. Their labels do not establish market equilibrium, authorization security, or a quantum no-go theorem. The governed scripts make no LLM calls, so hypothetical unconstrained-chat costs in older records are not a measured comparison.

For model integration, keep the proposer behind the same boundary: parse the proposed fields, check the declared carrier and operations, recompute the relevant property, and save the actual response. Consult the streaming harness as an implementation example; reviewing its adapter does not establish a live provider run. The harness accepts integers and exact rational strings as claim values and refuses a JSON float or Boolean before any scoring, since `Fraction(0.1)` is the binary expansion of the float rather than one tenth (code/tests/test_harness_exact_inputs.py).

8. Connections and next reading

The familiar idea of an untrusted producer supplying evidence to a checker appears in Necula's proof-carrying code. That work concerns proofs of a declared code-safety policy. It supplies context for separating proposal production and evidence checking; this tutorial does not claim its graph receipt is such a proof.

The minimization and lower-bound reasoning connect to linear-programming duality; see Boyd and Vandenberghe, Convex Optimization, Chapter 5. The series' graph mathematics paper develops the finite theory; Paper 004 explains native verification and evidence custody. The curriculum-design guide turns examples into lessons, and the review protocol explains how to audit a claimed result.

This guide supports a finite, inspectable proposal–check workflow. Its practical success criterion is that the reader can state the original instance, explain the failed proposal, distinguish residual concentration from claim replacement, and identify exactly what a fresh receipt records. General semantic correctness and model quality need their own evidence.

9. Limitations

The guide works one three-edge instance by hand and describes one scripted session. The package tests check the session table on the governor's exact Python enumeration path. For this revision the native-requesting tests were also run against a `solve_graph_phi1` executable built on Windows (x86_64-pc-windows-msvc, Rust 1.98.0) with `cargo build --locked -j 1` from the `omega_functional` crate, whose source is unchanged since the code baseline; with that executable and its SHA-256 (2b0379557993246cacb1ed8964f9f096964019ecc1c2fb781e6b18f2ca6c8a65) pinned, all 75 package tests passed and none was skipped. The dated review receipt records that run. The executable is not part of the deposit, so another reader must build and pin their own.

When a native executable is configured the governor uses it and propagates its refusals; otherwise it uses the enumeration, so a printed value does not by itself show which calculator produced it. The legacy scenario and streaming scripts are implementation examples, not measured comparisons.

This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against the manifests. The package tests and lint are executed by the dated review receipt recorded with this revision; that receipt lists each command, tool version, exit status, and tested scope. Historical receipts in this package keep their original dates and scope and are not current execution evidence.

References

- Necula, G. C. (1997). Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pp. 106–119. ACM. DOI: 10.1145/263699.263712.
- Boyd, S., and Vandenberghe, L. (2004). Convex Optimization. Cambridge University Press, Chapter 5. DOI: 10.1017/CBO9780511804441.
- Rundgren, A., Jordan, B., and Erdtman, S. (2020). JSON Canonicalization Scheme (JCS). RFC 8785 (Informational), RFC Editor, June 2020. DOI: 10.17487/RFC8785.